PROYECTO FINAL · SEGURIDAD INFORMÁTICA 2026

Fase 5

Simulación de ataques y tasa de detección

Evaluación de la capacidad de detección, prevención y contención del laboratorio Wazuh frente a las seis técnicas de ataque requeridas.

Plataforma	Wazuh 4.9.2 · Manager, Indexer y Dashboard
Agentes evaluados	`agent-linux` · `agent-db` · `agent-windows`
Periodo de ejecución	31/10/2026 – 02/11/2026
Técnicas	6 ataques requeridos por el enunciado
Evidencia	Registros del manager, telemetría de agentes y capturas del dashboard
Versión	1.0 (documento de entrega)

CONTENIDO DEL INFORME

01 Introducción y alcance
02 Resultados por técnica
03 Tasa de detección y falsos positivos
04 Hallazgos de infraestructura
05 Conclusiones y recomendaciones

> Informe standalone elaborado a partir de la evidencia conservada en `attacker/evidencia/`, `agent-windows-insider/` y los registros del manager. No se re-ejecutaron ataques para esta entrega.

Contenido

Resumen ejecutivo

Resultados en una página

1. Introducción y alcance

1.1 Objetivo
1.2 Entorno de laboratorio
1.3 Consideraciones metodológicas

2. Resultados por técnica

2.1 Escaneo de puertos (T1046)
2.2 Fuerza bruta SSH (T1110.001)
2.3 Inyección SQL (T1190)
2.4 Escalada local y persistencia en `sudoers` (T1548.003, `agent-db`)
2.5 Escalada de privilegios en Windows mediante BYOVD (T1068)
2.6 Exfiltración de datos (T1041 / T1571)

3. Tasa de detección y falsos positivos

3.1 Cálculo de la tasa
3.2 Falsos positivos y ruido

4. Hallazgos de infraestructura

5. Conclusiones y recomendaciones

Referencias

Resumen ejecutivo

Se ejecutaron contra el laboratorio real las seis técnicas de ataque requeridas: reconocimiento con `nmap`, fuerza bruta SSH con `hydra`, inyección SQL, modificación de `sudoers`, escalada de privilegios en Windows mediante un controlador vulnerable (BYOVD) y exfiltración de datos con `netcat`.

El laboratorio detectó correctamente 3 de las 6 técnicas (50,0 %), y las tres mediante una regla diseñada específicamente para la técnica: fuerza bruta SSH (`100001`), inyección SQL (`31103`, regla integrada de Wazuh) y exfiltración (`100014`). Excluyendo el escaneo de puertos, para el que no se configuró ninguna regla, la tasa sube a 3 de 5 (60,0 %).

La fuerza bruta SSH quedó contenida de extremo a extremo: la regla `100001` generó la alerta y el Active Response bloqueó la IP atacante durante 10 minutos exactos. La exfiltración por el puerto 4444 fue detectada por una regla específica de nivel 13. La inyección SQL fue detectada específicamente por `31103` ("SQL injection attempt"), una regla integrada de Wazuh; la regla propia del proyecto para esta técnica, `100021`, nunca llega a evaluarse, por una razón de decodificación documentada en la sección 2.3. Las dos brechas reales son la escalada de privilegios en Windows, bloqueada por el sistema operativo pero sin ninguna alerta en Wazuh, y la modificación de `sudoers` en `agent-db`, donde el módulo FIM no generó un solo evento en todo su historial. La causa más probable en ambos casos es el disco lleno del manager durante la ventana de pruebas, documentado en la sección 4.

Resultados en una página

#	Técnica	MITRE	Resultado del ataque	Detección
1	Escaneo de puertos	T1046	Éxito: `22/tcp` y `5432/tcp` confirmados	No esperada
2	Fuerza bruta SSH	T1110.001	Éxito: credencial válida en 21 s	Sí, específica (`100001` + Active Response)
3	Inyección SQL	T1190	Éxito manual; `sqlmap` con falso negativo	Sí, específica (`31103` integrada; `100021` propia, inalcanzable)
4	Modificación de `sudoers`	T1548.003	Éxito; revertido tras capturar evidencia	No (cero FIM en `agent-db`)
5	Escalada en Windows (BYOVD)	T1068	Bloqueada por el SO (código 1275)	No (cero alertas en Wazuh)
6	Exfiltración de datos	T1041 / T1571	Éxito: `/etc/passwd` completo	Sí, específica (`100014`, nivel 13)

1. Introducción y alcance

1.1 Objetivo

Medir la capacidad del laboratorio para detectar, contener o prevenir, mediante las reglas y módulos de Wazuh [3], las seis técnicas de ataque requeridas, mapeadas al catálogo MITRE ATT&CK [2] y evaluadas según las categorías de detección y manejo de incidentes descritas en [1] y [5], documentando la evidencia que sostiene cada resultado con trazabilidad hasta el registro del manager.

1.2 Entorno de laboratorio

Activo	Dirección	Función	Telemetría relevante
`agent-linux`	192.168.100.20	Servidor web Apache con `consultas.php`	`access.log`, `auth.log`, FIM, journald
`agent-db`	192.168.100.40	Servidor PostgreSQL y SSH	`auth.log`, `kern.log`, journald, FIM
`agent-windows`	192.168.100.30	Servidor Windows con registro de eventos	EventChannel, Security, CodeIntegrity
`wazuh-server`	192.168.100.10	Manager, indexador y dashboard	`alerts.json`, `archives.json`, `active-responses.log`
Kali	192.168.100.50	Estación atacante	Registros de ejecución y evidencia recolectada

1.3 Consideraciones metodológicas

Las pruebas se ejecutaron con autorización, sobre un laboratorio académico aislado. La fuerza bruta SSH usó una cuenta de prueba dedicada (`testuser`, con contraseña débil asignada a propósito) y una lista corta de contraseñas, en lugar de la cuenta `root` real y de `rockyou.txt` completo: así se evita dejar sin acceso administrativo a la máquina compartida durante la demostración y se mantiene el ejercicio enfocado en el mecanismo de detección. Las contraseñas obtenidas se omiten en este informe y los datos extraídos de la base de datos se identifican por fila, no por contenido nominal. Los cambios introducidos durante las pruebas (la regla de `sudoers`, los servicios de Windows) se revirtieron o eliminaron al finalizar.

2. Resultados por técnica

2.1 Escaneo de puertos (T1046)

El escaneo se dirigió contra `agent-db` con `nmap -sV -O`. Se confirmaron dos puertos abiertos: `22/tcp` (OpenSSH 10.0p2 Debian) y `5432/tcp` (PostgreSQL). La detección de sistema operativo fue inconclusa; la de servicios, exitosa. No se configuró ninguna regla para descubrimiento de servicios, por lo que no se esperaba alerta y no se produjo. Esta ausencia es una decisión de alcance, no una falla del motor, aunque deja al laboratorio sin visibilidad sobre la fase de reconocimiento.

Evidencia: `attacker/evidencia/01_portscan_20261102_1748.log`.

2.2 Fuerza bruta SSH (T1110.001)

El ataque se dirigió contra `agent-db`, cuenta `testuser`, con `hydra -l testuser -P wordlists/small_list.txt -f -t 4`. La corrida encontró la credencial válida en 21 segundos.

La regla `100001` (nivel 10, "SSH: fuerza bruta - más de 5 intentos fallidos en 60s") generó la alerta, y el Active Response `firewall-drop` insertó una regla `DROP` para la IP atacante en `agent-db`, verificada con `iptables -L INPUT`. El bloqueo duró 10 minutos exactos y expiró automáticamente. La Figura 1 muestra la regla `iptables` y el registro de `active-responses.log` que originó la acción.

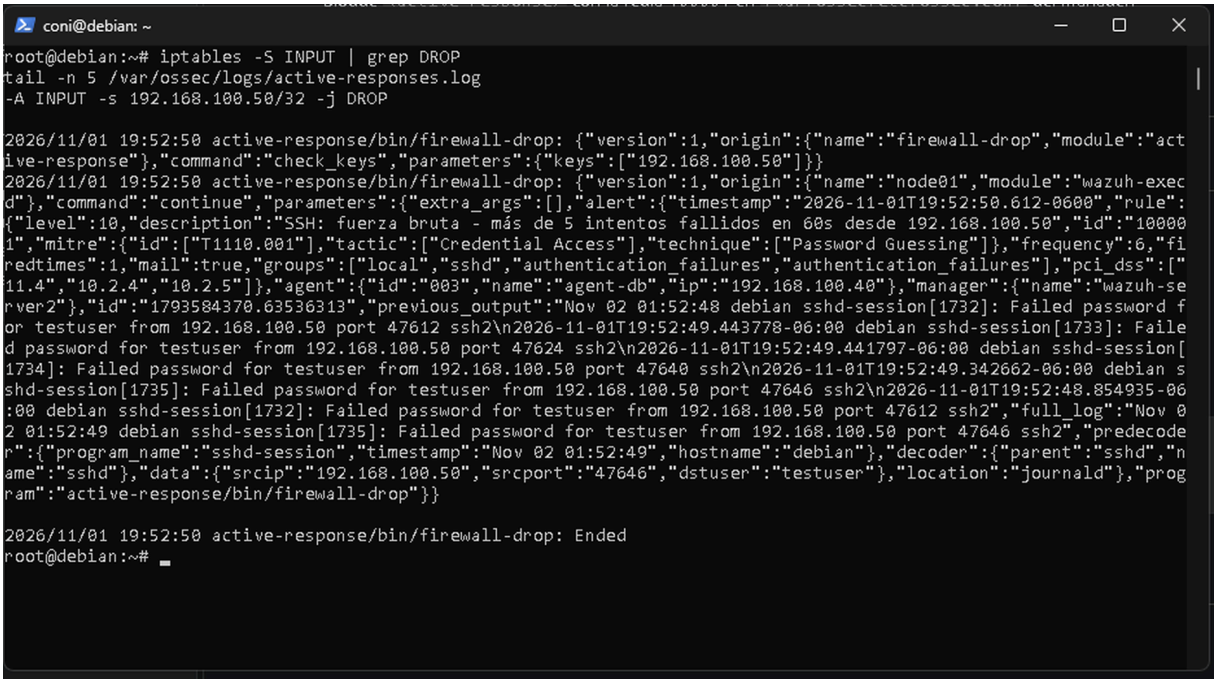


Figura 1. Contención de la fuerza bruta SSH: regla `DROP` insertada por `firewall-drop` y registro de `active-response` asociado a la regla `100001`.

Evidencia: `attacker/evidencia/02_bruteforce_success_20261101_2149.log`, `02_bruteforce_20261101_2230.log`, `02_bruteforce_20261101_2250.log`.

2.3 Inyección SQL (T1190)

El objetivo fue `agent-linux`, sobre el parámetro `busqueda` de `consultas.php`. El intento automático con `sqlmap` no confirmó la inyección: el parámetro rechaza cadenas de más de 100 caracteres en el propio código PHP, y los payloads de `sqlmap` suelen superar ese límite. El resultado "not injectable" es un falso negativo de la herramienta, no una prueba de que el endpoint fuera seguro [4][9].

Con payloads `UNION` de menos de 100 caracteres, la explotación manual obtuvo la versión del motor (PostgreSQL 17.11), el usuario de conexión (`admin`) y tres filas de la tabla `cliente`, identificadas por fila (1, 2, 3) sin reproducir su contenido nominal.

La inyección fue detectada de forma específica por la regla integrada de Wazuh `31103` ("SQL injection attempt", nivel 7, MITRE T1190, grupo `sql_injection`): se activó 121 veces el 1 de noviembre y 2 veces el 31 de octubre, contando tanto los payloads de `sqlmap` como la explotación manual. El siguiente es un registro real, extraído del manager:

```
timestamp:   2026-11-01T22:01:58.845-0600
rule.id:     31103 · level: 7 · "SQL injection attempt." · mitre: T1190
decoder:     web-accesslog
agent:       agent-linux (192.168.100.20)
full_log:    ...busqueda=(SELECT (CASE WHEN (5476=4144) THEN 'test'
             ELSE (SELECT 4144 UNION SELECT 7897) END)) HTTP/1.1" 500 229
             ... "sqlmap/1.8.5#stable (https://sqlmap.org)"
```

La regla propia del proyecto para esta técnica, `100021` ("Apache: posible inyección SQL"), nunca se activó, y la causa no es su patrón. Se verificó con `wazuh-logtest` sobre esta misma línea real que el decoder que procesa el log es el integrado `web-accesslog`, no el decoder personalizado `wz-apache-access` del que depende `100021` (a través de la regla base `100020`). Wazuh carga los decoders integrados antes que los

locales, y el integrado también reconoce el formato de estas líneas, así que siempre gana: `100020` nunca recibe el evento decodificado como espera, y `100021` queda inalcanzable sin importar su patrón. No es un problema de mayúsculas y minúsculas: la propia regla `31103`, con un patrón igual de sensible a mayúsculas, detectó el payload `UNION SELECT` en mayúsculas del registro citado arriba. El hallazgo se deja documentado tal como está desplegado en el manager.

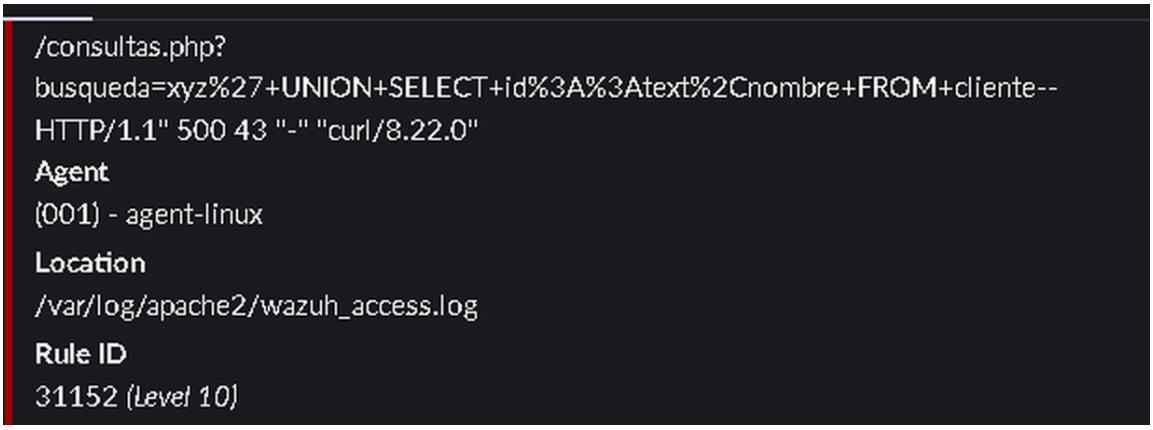


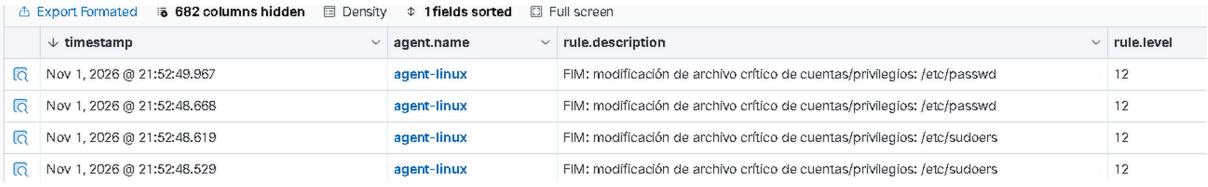
Figura 3. Alerta genérica `31152` (nivel 10) por error HTTP 500, registrada junto con la detección específica de la regla `31103`.

Evidencia: `attacker/evidencia/03_sqlmap_20261031_2340.log`, `03b_sqli_manual_20261101_2313.log`; alerta `31103` verificada en `/var/ossec/logs/alerts/2026/Nov/ossec-alerts-01.json` del manager.

2.4 Escalada local y persistencia en `sudoers` (T1548.003, `agent-db`)

Un chequeo inicial (`sudo -l`, búsqueda de binarios SUID) sobre la cuenta `testuser` no arrojó hallazgos explotables. Para el ejercicio se configuró a propósito una regla de `sudoers` mal pensada (`NOPASSWD` sobre `/usr/bin/find`); la explotación con GTFOBins (`sudo find . -exec /bin/sh \; -quit`) produjo una shell interactiva de root [6]. La técnica correcta es T1548.003 (abuso de un mecanismo legítimo de control de privilegios), no T1068, que se reserva para la explotación de una vulnerabilidad de software o kernel real.

Desde esa shell se agregó `testuser ALL=(ALL) NOPASSWD:ALL` a `/etc/sudoers`, como mecanismo de persistencia. El grupo `db` de Wazuh, al que pertenece `agent-db`, tiene FIM en tiempo real sobre `/etc/passwd`, `/etc/shadow`, `/etc/sudoers` y `/etc/sudoers.d` (regla `100002`, nivel 12), ya activo antes del ataque. No se produjo ninguna alerta: la revisión de `ossec-alerts-01.json` muestra cero eventos `syscheck` para `agent-db` en todo su historial. El mismo mecanismo sí funciona en `agent-linux` (Figura 4), lo que descarta una regla mal escrita y orienta la causa hacia el disco lleno del manager durante esa ventana (sección 4). El cambio se revirtió tras capturar la evidencia.


Export Formated | 682 columns hidden | Density | 1 fields sorted | Full screen

timestamp	agent.name	rule.description	rule.level
Nov 1, 2026 @ 21:52:49.967	agent-linux	FIM: modificación de archivo crítico de cuentas/privilegios: /etc/passwd	12
Nov 1, 2026 @ 21:52:48.668	agent-linux	FIM: modificación de archivo crítico de cuentas/privilegios: /etc/passwd	12
Nov 1, 2026 @ 21:52:48.619	agent-linux	FIM: modificación de archivo crítico de cuentas/privilegios: /etc/sudoers	12
Nov 1, 2026 @ 21:52:48.529	agent-linux	FIM: modificación de archivo crítico de cuentas/privilegios: /etc/sudoers	12

Figura 4. Eventos FIM de la regla `100002` (nivel 12) sobre `/etc/passwd` y `/etc/sudoers` en `agent-linux`; el mismo mecanismo no generó ningún evento para `agent-db`.

Evidencia: `attacker/evidencia/04b_privesc_find_20261101_2354.log`.

2.5 Escalada de privilegios en Windows mediante BYOVD (T1068)

La técnica se ejecutó con Atomic Red Team contra `agent-windows` [7]. Requiere privilegios de Administrator local: sin esa elevación, `sc.exe create` es rechazado por control de acceso antes de llegar al kernel.

`sc.exe create` registró el servicio `iqvw64e.sys` sin incidentes. `sc.exe start` falló con el código **1275 (`ERROR_DRIVER_BLOCKED`)**: el Microsoft Vulnerable Driver Blocklist de Windows Server 2022 (Code Integrity / WDAC) impidió la carga antes de que el controlador alcanzara el kernel [8]. No hubo caída ni inestabilidad del sistema; la verificación posterior mostró el servicio en estado `STOPPED` con `WIN32_EXIT_CODE 1275`, confirmando que el bloqueo fue persistente.

El Service Control Manager registró el evento **7045** ("a service was installed"). El log `Microsoft-Windows-CodeIntegrity/Operational` solo mostró eventos genéricos de carga de política (ID 3085), sin un evento diferenciado de bloqueo por intento. La búsqueda de `iqvw64e` en `alerts.json` y `archives.json` del manager no arrojó coincidencias, pese a que `agent-windows` tiene más de 10 000 alertas registradas y a que la regla `100005` sí se activó en la misma ventana, lo que descarta un problema de recolección. El evento existe en el log local de Windows y el canal funciona; falta la regla que lo convierta en alerta. Es la brecha de detección más concreta del informe.

Durante la prueba se encontró Windows Defender temporalmente deshabilitado, sin relación con el bloqueo del controlador (que depende de Code Integrity/WDAC, no del antivirus); se reactivó y se confirmó `RealTimeProtectionEnabled: True`. Los registros de servicio quedaron eliminados al finalizar.

Evidencia: `agent-windows-insider/T1068_staging_evidencia_20261102.log`.

2.6 Exfiltración de datos (T1041 / T1571)

Se abrió un listener en Kali sobre el puerto 4444 y se envió `/etc/passwd` desde `agent-db`; el archivo llegó completo. La regla `100014` (nivel 13) disparó la alerta "Red: conexión SALIENTE al puerto 4444 hacia 192.168.100.50 (posible reverse shell)" dos veces a las 22:06:38, una por `journald` y otra por `/var/log/kern.log`, porque ambas fuentes capturaron el mismo evento.

La regla de logging de `iptables` se corrigió de la cadena `INPUT` a `OUTPUT` durante la verificación, ya que la conexión relevante es saliente (Figura 6); la alerta `100014` corresponde a la regla ya corregida.

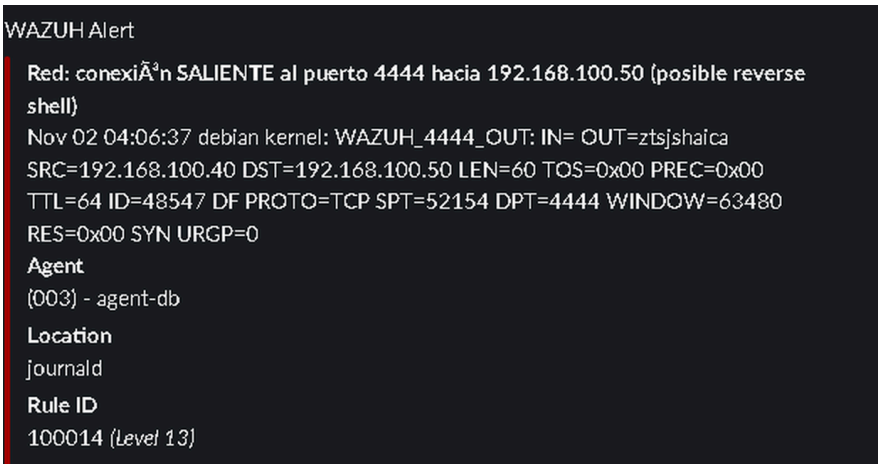


Figura 2. Alerta `100014` (nivel 13) por conexión saliente al puerto 4444 hacia la estación atacante.

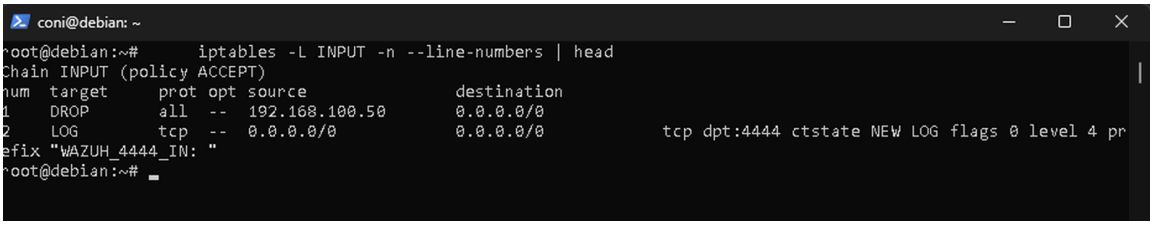


Figura 6. Regla de logging corregida de la cadena `INPUT` a `OUTPUT` para capturar la conexión saliente al puerto 4444.

Evidencia: `attacker/evidencia/exfil_rec.txt`.

3. Tasa de detección y falsos positivos

3.1 Cálculo de la tasa

La tasa de detección se calcula como (alertas correctas / ataques ejecutados) × 100 %. Sobre los seis ataques requeridos, el laboratorio generó una alerta correcta y específica en 3 casos (fuerza bruta SSH, inyección SQL, exfiltración): **3/6 = 50,0 %**. El escaneo de puertos no tenía detección esperada; excluyéndolo, la tasa sobre los cinco ataques restantes es **3/5 = 60,0 %**.

Las tres detecciones son específicas, pero no las tres son obra del ruleset propio del proyecto: la fuerza bruta SSH y la exfiltración se detectan con reglas escritas para este laboratorio (`100001`, `100014`); la inyección SQL se detecta con una regla integrada de Wazuh (`31103`) que ya cubría el caso, mientras que la regla propia (`100021`) nunca llega a evaluarse (sección 2.3). Vale la pena recordar esto para la defensa: el 50,0 % de detección no depende por completo del trabajo de regla propia del equipo. Por separado, el laboratorio previno 2 de 2 intentos de escalada de privilegios (Windows por el sistema operativo, Linux por reversión manual tras capturar evidencia), sin que ninguna de esas acciones sustituya a la detección en Wazuh.

3.2 Falsos positivos y ruido

No se registró ningún falso positivo propio durante las pruebas. Se corrigió un caso de diagnóstico erróneo: un bloqueo HTTP 405 se atribuyó inicialmente a una regla de WAF o `fail2ban`, pero la causa real fue un cambio de método HTTP en la aplicación durante la sesión.

La máquina `misp-agent-linux` (192.168.100.11) genera intentos de fuerza bruta contra `root` de forma independiente a estas pruebas; al revisar el dashboard conviene filtrar por IP de origen para no confundirlos con el ejercicio. La integración MISP (regla `100050`) marcó la IP atacante como indicador de compromiso (Figura 7); es un evento de enriquecimiento, no una detección del ataque, y en este caso es circular porque la IP es la propia estación atacante del ejercicio.

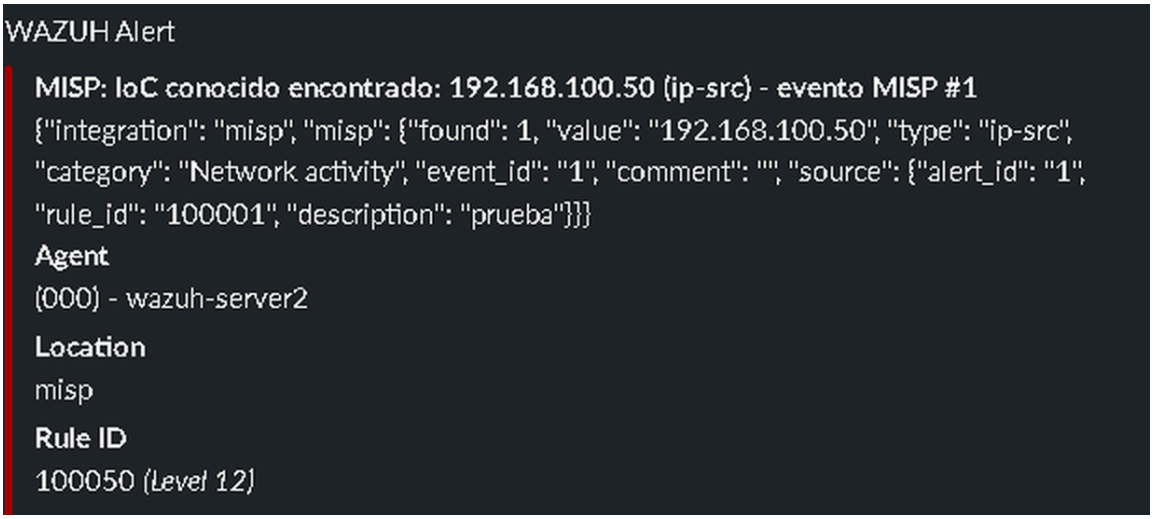


Figura 7. Alerta `100050` de enriquecimiento MISP sobre la IP atacante; no constituye una detección del ataque.

4. Hallazgos de infraestructura

Tres condiciones del manager afectaron la consistencia de los resultados durante la ventana de pruebas:

1. El `wazuh-logcollector` dejó de leer el journal de systemd durante un tramo, deteniendo la generación de alertas; coincide con los periodos en que el Active Response de SSH no se disparó pese a cumplirse las condiciones de la regla.
2. El disco del manager llegó al 100 %, principalmente por la caché de `/var/ossec/queue/vd_updater` (unos 27 GB acumulados). Es la causa más probable de la ausencia de eventos FIM en `agent-db`.
3. La regla de logging del puerto 4444 estaba inicialmente en la cadena `INPUT`; se corrigió a `OUTPUT` durante la verificación (sección 2.6).

5. Conclusiones y recomendaciones

El laboratorio detecta correctamente 3 de las 6 técnicas requeridas (50,0 %), las tres con una regla específica. Los controles reales funcionan: el Active Response de SSH contuvo el ataque de extremo a extremo, la regla de exfiltración identificó el reverse shell en el puerto 4444, y la inyección SQL fue clasificada correctamente como tal por una regla integrada de Wazuh. Las dos brechas, Windows sin alerta pese al bloqueo del sistema operativo y `agent-db` sin un solo evento FIM, comparten una causa probable común: el disco lleno del manager durante la ventana de pruebas.

Una acción cierra la brecha de `sudoers`: liberar espacio en el disco del manager y resincronizar `agent-db` para confirmar que FIM reporta correctamente. La brecha de Windows no corresponde a ninguna de las cinco reglas personalizadas que pide el enunciado; como mejora opcional, podría agregarse una regla local que correlacione el evento 7045 con la instalación de servicios de kernel, para que el bloqueo de controladores también quede reflejado en una alerta. Una acción adicional, de menor prioridad porque la cobertura ya existe vía `31103`, es decidir si vale la pena reconectar la regla propia `100021` a la cadena de decoders que realmente procesa el log de Apache (enganchándola a la regla base integrada en vez de al decoder personalizado), o si se retira por redundante.

Referencias

[1] NIST, *Guide to Intrusion Detection and Prevention Systems (IDPS)*, NIST Special Publication 800-94. Gaithersburg, MD, USA: National Institute of Standards and Technology, Feb. 2007.

[2] MITRE, *ATT&CK Enterprise Matrix*. [Online]. Available: https://attack.mitre.org/. [Accessed: 04-Nov-2026].

[3] Wazuh, *Wazuh Documentation 4.9*. [Online]. Available: https://documentation.wazuh.com/. [Accessed: 04-Nov-2026].

[4] OWASP Foundation, *OWASP Top 10:2021*. [Online]. Available: https://owasp.org/Top10/. [Accessed: 04-Nov-2026].

[5] NIST, *Computer Security Incident Handling Guide*, NIST Special Publication 800-61 Rev. 2. Gaithersburg, MD, USA: National Institute of Standards and Technology, Aug. 2012.

[6] GTFOBins, *GTFOBins: find*. [Online]. Available: https://gtfobins.github.io/gtfobins/find/. [Accessed: 04-Nov-2026].

[7] Red Canary, *Atomic Red Team*. [Online]. Available: https://github.com/redcanaryco/atomic-red-team. [Accessed: 04-Nov-2026].

[8] Microsoft, *Microsoft Vulnerable Driver Blocklist*. [Online]. Available: https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/design/microsoft-recommended-driver-block-rules. [Accessed: 04-Nov-2026].

[9] MITRE, *CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')*. [Online]. Available: https://cwe.mitre.org/data/definitions/89.html. [Accessed: 04-Nov-2026].